

App Dev Project Report

1. Student Details

Name: Shorya

Roll Number: 25f2007158

Email: 25f2007158@ds.study.iitm.ac.in

About Me: I am a student in the IIT Madras BS Degree program with a keen interest in developing full-stack web applications that solve real-world organizational challenges.

2. Project Details

Project Title: Placement Portal Application (PPA)

Problem Statement: To design and build a centralized web-based platform that streamlines the recruitment process by connecting students, companies, and administrators. The goal is to manage job postings, student applications, and placement tracking in a role-based secure environment.

Approach: The application is built using the Flask framework with a focus on Role-Based Access Control (RBAC). It features three distinct dashboards:

- **Admin:** To manage users, approve companies, and oversee job postings.
- **Company:** To post job drives and manage student applications.
- **Student:** To build profiles, upload resumes, and track application statuses.

3. AI/LLM Declaration

I used Gemini (Google) to assist in resolving Git version control conflicts (merging remote and local repositories), setting up GitHub Milestones, and debugging Bootstrap CSS for responsive image alignment.

- The extent of AI/LLM usage is approximately **10%**, primarily focused on environment configuration, troubleshooting Git errors, and frontend styling.
- All core application logic, database schema design, and Flask route implementation were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	ORM for SQLite database management
Jinja2	Template engine for dynamic HTML rendering
Bootstrap 5	Responsive UI components and layout styling
SQLite	Lightweight database for storing portal data

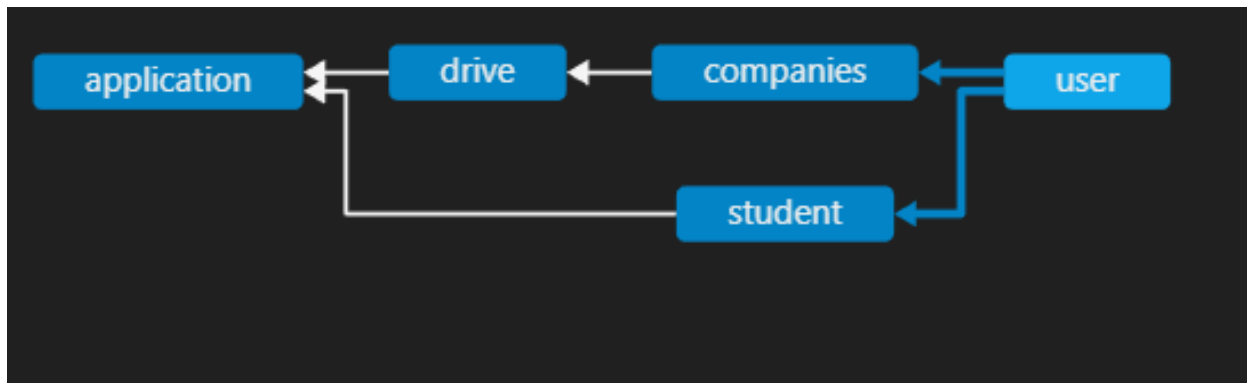
5. Database Schema / ER Diagram

Tables:

- **User:** Central authentication table storing username, email, password, and role.
- **Student:** Linked to User; stores academic profile details like department, skills, projects, and experience.
- **Companies:** Linked to User; contains company status (for Admin approval tracking) and description.
- **Drive:** Represents job postings created by companies; includes job_title, salary, eligibility, and deadline.
- **Application:** A junction table connecting Student and Drive to track specific job applications and their current status.

Relationships:

- **User to Student/Companies:** One-to-One (via user_id foreign key).
- **Companies to Drive:** One-to-Many (One company can host multiple placement drives).
- **Student to Application:** One-to-Many (One student can apply for multiple drives).
- **Drive to Application:** One-to-Many (One drive can receive multiple student applications).



6. API Resource Endpoints

The application follows a RESTful routing structure where standard HTML forms interact with backend endpoints. While a separate JSON-based API was not implemented for this version, the following core functional routes handle data processing and state changes:

Endpoint	Method	Description
1) /login	POST	Processes credentials and establishes a secure user session.
2) /register	POST	Handles multi-table insertion for User and Student/Companies.
3) /create_drive	POST	Commits new job data to the Drive table from a Company form.
4)/applying_to_drive/<int:id>	POST	Links a Student to a Drive within the Application table.
5) /approve_company/<id>	POST	Admin route to update Companies.status to 'approved'.

YAML API Definition File:

Included as `api.yaml`. This file serves as the architectural design for future API integration, mapping the existing database models to standard OpenAPI endpoints.

7. Architecture and Features

Architecture Overview:

- `app.py`: The entry point for the Flask application.
- `.database`: Module where the SQLAlchemy db instance is initialized.

- **models.py:** Defines the relational schema (User, Student, Companies, Drive, Application).
- **templates/:** HTML files for role-specific dashboards.
- **static/:** Assets for UI styling and profile data.

Implemented Features:

- **Pending Approval System:** Companies are initialized with a pending status, requiring Admin intervention in the Companies table before they can access the dashboard.
- **Dynamic Drive Management:** Use of the Drive model to allow companies to post full job descriptions, salary details, and eligibility criteria.
- **Relational Tracking:** Using db.relationship and backref to seamlessly connect student applications with company job drives.

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1OZ8zBtNyL4zt3P57a2rBUBnqQK1fUfao/view?usp=sharing>